HTTP Verb	Path	Controller#Action	Used for
GET	/magazines/:magazine_id/ads/:id/edit	ads#edit	return an HTML form for editing an ad belonging to a specific magazine
PATCH/PUT	/magazines/:magazine_id/ads/:id	ads#update	update a specific ad belonging to a specific magazine
DELETE	/magazines/:magazine_id/ads/:id	ads#destroy	delete a specific ad belonging to a specific magazine

This will also create routing helpers such as `magazine_ads_url` and `edit_magazine_ad_path`. These helpers take an instance of Magazine as the first parameter (`magazine_ads_url(@magazine)`).

2.7.1 Limits to Nesting

You can nest resources within other nested resources if you like. For example:

```
resources :publishers do
  resources :magazines do
    resources :photos
  end
end
```

Deeply-nested resources quickly become cumbersome. In this case, for example, the application would recognize paths such as:

```
/publishers/1/magazines/2/photos/3
```

The corresponding route helper would be `publisher_magazine_photo_url`, requiring you to specify objects at all three levels. Indeed, this situation is confusing enough that a popular article by Jamis Buck proposes a rule of thumb for good Rails design:
Resources should never be nested more than 1 level deep.

2.7.2 Shallow Nesting

One way to avoid deep nesting (as recommended above) is to generate the collection actions scoped under the parent, so as to get a sense of the hierarchy, but to not nest the member actions. In other words, to only build routes with the minimal amount of information to uniquely identify the resource, like this:

```
resources :articles do
  resources :comments, only: [:index, :new, :create]
end
resources :comments, only: [:show, :edit, :update, :destroy]
```

This idea strikes a balance between descriptive routes and deep nesting. There exists shorthand syntax to achieve just that, via the `:shallow` option:

```
resources :articles do
  resources :comments, shallow: true
end
```

This will generate the exact same routes as the first example. You can also specify the `:shallow`option in the parent resource, in which case all of the nested resources will be shallow:

```
resources :articles, shallow: true do
  resources :comments
```